Simulating hematopoietic phylogeny for SCM validation

Michael Tassia

2026-07-28

Overview

The objective of this notebook is to document our validation strategy for applying stochastic character mapping as a technique for reconstructing somatic genotypes given a phylogenetic tree, substitution model, and a data set of observed genotype states at the tips (see *Bollback 2006* and *Revell 2024*). As a ground-truth somatic phylogeny data set does not yet exist, we will leverage simulation as a framework for validation.

This notebook will first describe the approach, key functions, and expected outcomes. Afterwards, additional layers of technical and biological variation will be introduced, including:

- Sample size
- Sequencing depth
- Missing data
- ISM-violation rate

Simulating hematopoiesis

We use the publicly available `rsimpop` package developed by Nick Williams at the Sanger Institute to simulate hematopoiesis under a birth-death model. This package implements the Gillepsie algorithm (see here for basic overview) to simulate cellular compartments under a distribution of user-defined parameters and is described in the supplementary information of *Mitchell et al. 2022*.

Here, we use `rsimpop` with parameters set following empirical estimates from the literature. Fitness coefficients manifest as increased symmetric division rates, and polygenic effects are assumed to be additive.

The libraries and functions used in this notebook are tracked in `simphy_functions.R`.

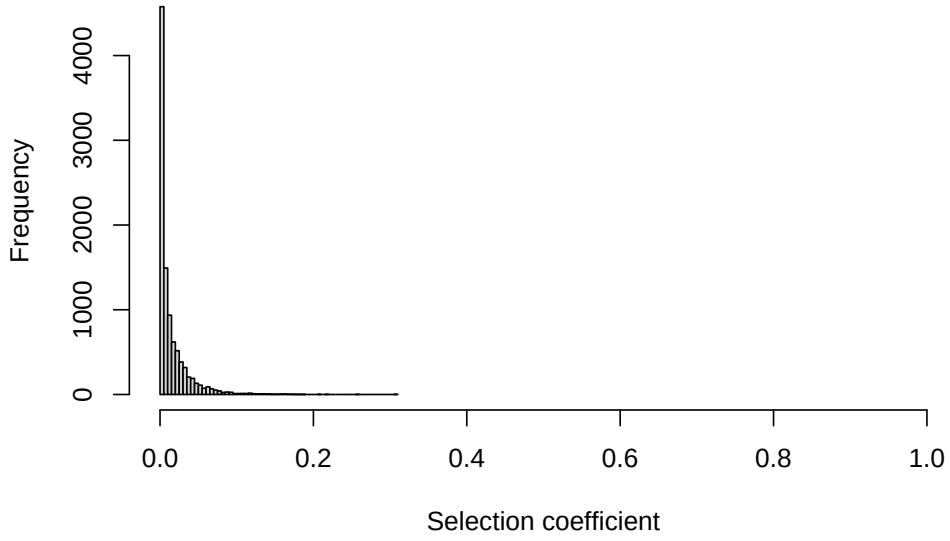
```
suppressMessages(
  source("simphy_functions.R")
)
```

Distribution of fitness effects (DFE)

`rsimpop` encodes fitness as increased rates of symmetric division (i.e., birth) for clones bearing a driver mutation. Here, we draw fitness effects from a gamma distribution using the parameters described in Extended Data Fig 12 of *Mitchell et al. 2022*.

```
# DFE function
fitnessFn <- genGammaFitness(shape = 0.47, rate = 34)

# Plot DFE
hist(sapply(1:1e4, function(x) exp(fitnessFn()) - 1),
```

```
    breaks = 100, xlim = c(0, 1),
    xlab = "Selection coefficient", main = "")
```



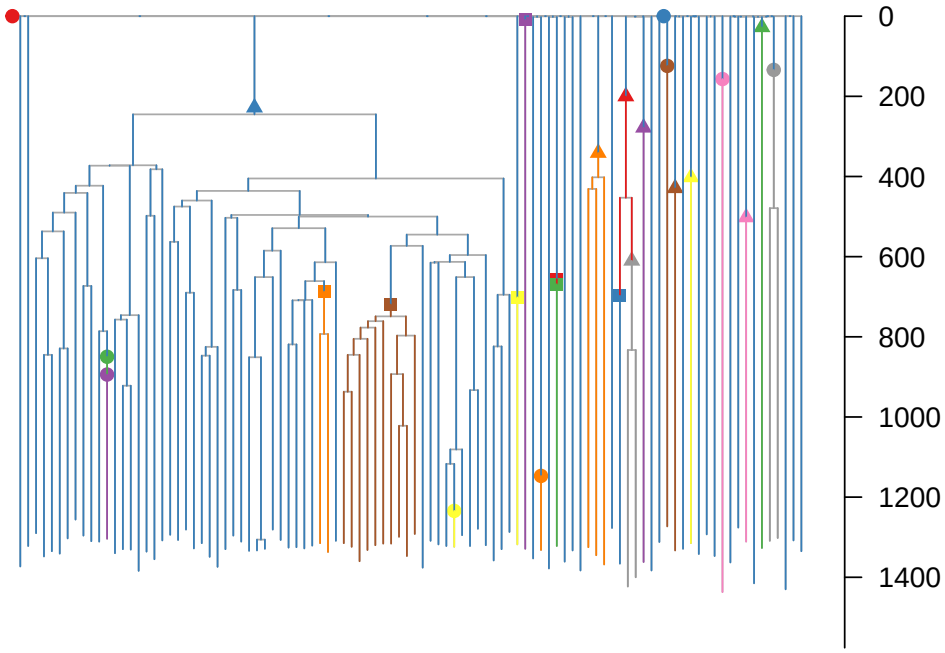
Simulate HSC population

Using the DFE above, we can simulate hematopoiesis with stochastic introduction of driver mutations using the `run_driver_process_sim()` wrapper in `rsimpop`. This function has several important parameters which will be defined below:

- `simpop = NULL`: Default setting; do not use a pre-existing simpop object
- `initial_division_rate = 0.1`: Default setting; parameter estimate for HSCs during development; only effects early population growth phase.
- `final_division_rate = 1/365`: Default setting; parameter estimate for HSCs during adulthood (concordant with the ABC estimates in *Mitchell et al. 2022*).
- `target_pop_size = 1e5`: Default setting; mean parameter estimate for HSC population size at adulthood (concordant with the ABC estimates in *Mitchell et al. 2022*).
- `nyears = 50`: User-defined age at collecting data. Validation will integrate across a distribution of ages.
- `fitness = fitnessFn`: User-defined fitness function defined above
- `drivers_per_cell_per_day = 2.0e-3 / 365`: User-defined count of driver mutations entering the population per day. This is the mean value estimated by ABC in *Mitchell et al. 2022*.

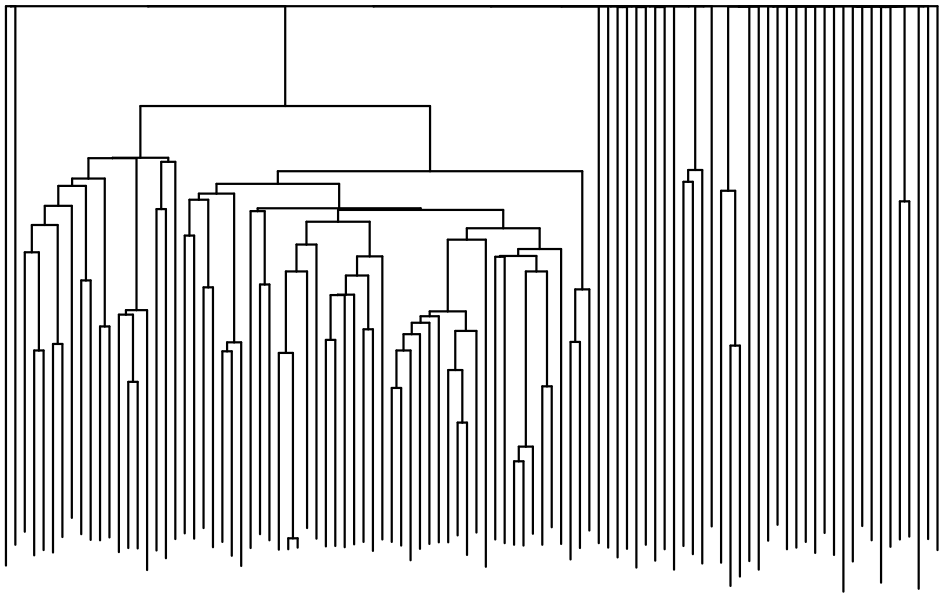
```
# Simulation seed
SEED <- 1212121
initSimPop(SEED, bForce = TRUE)

# Run rsimpop
sim <- run_driver_process_sim(
  simpop                  = NULL,
  initial_division_rate   = 0.1,
  final_division_rate     = 1/365,
  target_pop_size         = 1e5,
  nyears                  = 80,
  fitness                 = fitnessFn,
```

```
  drivers_per_cell_per_day = 2.0e-3 / 365
)

# Sample 100 cells from the HSC population
st <- get_subsampled_tree(sim, 100)

# Re-scale the phylogeny by mutations
st_mut <- get_elapsed_time_tree(st,
                                mutrateperdivision = 0,
                                backgroundrate = 16.8/365)

# Plot driver events on the phylogeny
plot_tree_events(st_mut,
                 cex.label = 0,
                 legpos = NULL)
```



Last before creating a mutation matrix that'll ultimately be used to approximate a VCF-like data structure, we need to extract the `phylo` object from the `simpop` output and remove the artificial out group (*s1*).

```
# Remove s1 from phylogeny
tr <- get_phylo_object(st_mut) %>% drop.tip("s1")

# Plot phylo
ggtree(tr,
       ladderize = FALSE) +
  layout_dendrogram()
```



Simulate mutation data

Because `rsimpop` tracks mutation origin alongside the "ground truth" simulated history, we can use these mutations to reconstruct a VCF-like structure that can later be used for SCM.

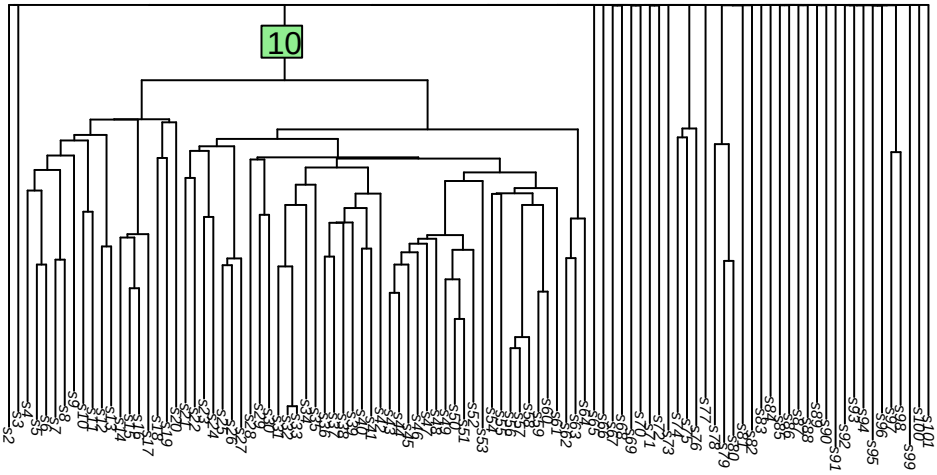
```
# Extract phyletic patterns per mutation
mut_mat <- create_mut_df(tr)

# Preview mutation matrix
mut_mat[1:5, 1:5]
```

```
  edge s2 s3 s4 s5
1    8  1  0  0  0
2    8  1  0  0  0
3    8  1  0  0  0
4    8  1  0  0  0
5    8  1  0  0  0
```

The `G` object above is a matrix of 0s and 1s, where rows correspond to mutations and columns correspond to sampled cells. A value of 1 indicates that the mutation is present in the cell, while a value of 0 indicates that it is absent. An additional column is included `G[,1]` to track the edge on which the mutation arose – this is important for tracking accuracy of SCM later.

For example:

```
# Plot tree with edge labels
plot.phylo(tr,
           show.tip.label = TRUE,
           direction = "downwards",
           cex = 0.5)
edgelabels(edge = 10)
```




```
# For the first row of G where G[,1] == 10, print columns where mutation is present
ingroup <- mut_mat[mut_mat[,1] == 10,] %>%
  head(n = 1) %>%
  colnames(.)[as.vector(.) == 1] %>%
  colnames()

cat(paste("Mutation events on edge 10 are inherited by the following tips:\n",
          ingroup[1],
          "to",
          ingroup[length(ingroup)]))
```

```
Mutation events on edge 10 are inherited by the following tips:
 s4 to s64
```

From this, we can use the `simulate_depth_and_ad` function to simulate sequencing data. Default parameters are set such that they mimic the structure of clean data described in both *Mitchell et al. 2022* and Coorens et al. 2025.

- `G`: Matrix output by `create_mut_df()`
- `D_bar = 20`: Target mean sequencing depth across the whole matrix
- `min_cov = 6`: Lower depth bound a site must clear to pass QC (Sequoia-style min_cov)
- `max_cov = 250`: Upper depth bound (excludes repeat/mapping-artifact loci)
- `site_sdlog = 0.5`: Log-scale spread of per-site coverage bias (GC/mappability)
- `sample_sdlog = 0.3`: Log-scale spread of per-sample library depth variation
- `theta = 8`: Negative-binomial dispersion for residual (post-QC) depth noise
- `rho_max = 0.1`: Beta-binomial overdispersion ceiling a site must clear (Sequoia snv_rho)
- `site_conc_shape = 4`: Gamma shape for the distribution of per-site VAF concentration
- `site_conc_rate = 0.3`: Gamma rate for the distribution of per-site VAF concentration
- `error_rate = 0.002`: Sequencing/mapping error rate (alt reads at true hom-ref sites)
- `dropout_rate = 0.00`: $P(\text{complete allelic dropout} | \text{truly heterozygous site})$

```
sim_reads <- simulate_DP_and_AD(G = mut_mat)
glimpse(sim_reads)
```

```
List of 7
```

```
 $ DP           : num [1:89503, 1:100] 14 20 38 28 18 48 8 11 15 11 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:100] "s2" "s3" "s4" "s5" ...
 $ AD           : int [1:89503, 1:100] 6 9 15 12 6 26 5 6 4 8 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:100] "s2" "s3" "s4" "s5" ...
 $ site_factor  : num [1:89503] 1.104 0.502 1.475 1.332 0.614 ...
 $ sample_factor: num [1:100] 1.03 1.128 0.826 1.22 1.25 ...
 $ site_conc    : num [1:89503] 10.8 15.6 19.2 12.5 15.1 ...
 $ site_rho     : num [1:89503] 0.085 0.0601 0.0494 0.0742 0.0621 ...
 $ dropout      : logi [1:89503, 1:100] FALSE FALSE FALSE FALSE FALSE FALSE ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:100] "s2" "s3" "s4" "s5" ...
```

From the `sim_reads` list output, the `AD` and `DP` data frames contain the simulated read data at each site for each sample. Using these values, we can approximate Phred-scaled genotype likelihoods following a structure similar to the fixed-error-rate GATK model (DePristo et al. 2011).

```
sim_genotypes <- simulate_GQ_PL_GT(AD = sim_reads$AD,
                                   DP = sim_reads$DP)
glimpse(sim_genotypes)
```

```
List of 5
 $ GQ : num [1:89503, 1:100] 99 99 99 99 99 99 99 99 99 99 ...
 $ PL0: num [1:89503, 1:100] 255 255 255 255 255 255 255 255 255 255 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:100] "s2" "s3" "s4" "s5" ...
 $ PL1: num [1:89503, 1:100] 0 0 0 0 0 0 0 0 0 0 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:100] "s2" "s3" "s4" "s5" ...
 $ PL2: num [1:89503, 1:100] 255 255 255 255 255 255 255 255 255 255 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:100] "s2" "s3" "s4" "s5" ...
 $ GT : chr [1:89503, 1:100] "0/1" "0/1" "0/1" "0/1" ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : NULL
  .. ..$ : chr [1:100] "s2" "s3" "s4" "s5" ...
```

Finally, we will sample sites from the human genome and assign these to the records held in `sim_genotypes`. These sites will then be threaded into a function to create a VCF-like structure.

```
mut_table <- sample_mutation_loci(nrow(sim_genotypes$GT))
vcf_df <- build_vcf_df(G = mut_mat,
                       DP = sim_reads$DP,
                       AD = sim_reads$AD,
                       gl = sim_genotypes,
                       edges = mut_mat$edge)

head(vcf_df[, 1:10])
```

```
  CHROM       POS ID REF ALT QUAL FILTER   INFO         FORMAT
1 chr18  61081263  .   A   T    .   PASS EDGE=8 GT:DP:AD:GQ:PL
2 chr10  97437719  .   T   C    .   PASS EDGE=8 GT:DP:AD:GQ:PL
3  chr2 177478383  .   T   A    .   PASS EDGE=8 GT:DP:AD:GQ:PL
4 chr16  76402884  .   T   A    .   PASS EDGE=8 GT:DP:AD:GQ:PL
5  chr3 163880494  .   T   C    .   PASS EDGE=8 GT:DP:AD:GQ:PL
6  chr8  76301790  .   A   T    .   PASS EDGE=8 GT:DP:AD:GQ:PL
                        s2
1   0/1:14:8,6:99:255,0,255
2  0/1:20:11,9:99:255,0,255
3 0/1:38:23,15:99:255,0,255
4 0/1:28:16,12:99:255,0,255
5  0/1:18:12,6:99:255,0,255
6 0/1:48:22,26:99:255,0,255
```